

Wprowadzenie do obliczeń w programie MATLAB

Bartosz Borucki, Maciej Szpindler

Interdyscyplinarne Centrum Modelowania
Matematycznego i Komputerowego
Uniwersytet Warszawski

10 marca 2006



Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM



Plan prezentacji

- 1 **Wstęp do MATLAB'a**
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 **Podstawowe problemy matematyczne**
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 **Toolbox-y**
- 8 **MATLAB w ICM**



- MATLAB jest zintegrowanym środowiskiem programistyczno-obliczeniowym
- Nazwa pochodzi od *MATrix LABoratory*
- Narzędzie służące do obróbki i analizy danych macierzowych
- Rozbudowane narzędzie o szerokiej funkcjonalności
- Optymalizacja pod kątem kodu *wektorowego i macierzowego*

- Obliczenia numeryczne
- Opracowywanie algorytmów
- Akwizycja danych
- Modelowanie i symulacja
- Analiza danych
- Wizualizacja
- Budowanie aplikacji wraz z projektowaniem interfejsów graficznych



- Interfejs graficzny
- Środowisko użytkownika
- Szybka implementacja własnych algorytmów macierzowych
- Optymalizacja obliczeniowa na działania macierzowe
- Dodatkowe wyspecjalizowane pakiety - tzw. *Toolbox'y*
- Możliwość samodzielnego rozszerzania

Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc**
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM



Podstawowy interfejs MATLAB'a - środowisko graficzne:

Run MATLAB

Interfejs użytkownika

- okno poleceń (Command Window)
- bieżący katalog (Current Directory)
- historia poleceń (Command History)
- przestrzeń robocza (Workspace)

Podstawowy interfejs MATLAB'a - środowisko graficzne:

Run MATLAB

Interfejs użytkownika

- okno poleceń (Command Window)
- bieżący katalog (Current Directory)
- historia poleceń (Command History)
- przestrzeń robocza (Workspace)

Podstawowy interfejs MATLAB'a - środowisko graficzne:

Run MATLAB

Interfejs użytkownika

- okno poleceń (Command Window)
- bieżący katalog (Current Directory)
- historia poleceń (Command History)
- przestrzeń robocza (Workspace)

Podstawowy interfejs MATLAB'a - środowisko graficzne:

Run MATLAB

Interfejs użytkownika

- okno poleceń (Command Window)
- bieżący katalog (Current Directory)
- historia poleceń (Command History)
- przestrzeń robocza (Workspace)

Korzystanie z Help'a

- Podręczna pomoc (Help), dostępna w trybie off-line Show Help
- Obejmuje podstawowe środowisko i Toolbox'y
- Bardzo dobrze zorganizowana
- Zawiera opisy poszczególnych funkcji i przykładowe zastosowania
- Wyszukiwarka oraz index
- Możliwość uruchamiania dołączonych programów demonstracyjnych



Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em**
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM



Korzystanie z okna poleceń:

Command Window

```
>>
```

Wprowadzanie komend i wyrażeń:

Command Window

```
>> exp(i*pi) + 1  
ans =  
0 + 1.2246e-16i
```

- Tryb graficzny i tekstowy
- Znaczenie ; na końcu linii
- **TAB** pozwala uzupełniać nazwy funkcji i zmiennych
- Wielkość liter jest rozróżniana
- W trybie graficznym prawy przycisk myszy
- Polecenie **clc** czyści okno
- **help**, **lookfor**

Filozofia MATLAB'a

Idea: w MATLAB każdy obiekt jest macierzą

Dane (macierze) mogą być:

- jawnie podane
- wynikiem działania lub funkcji
- wczytane ze źródła zewnętrznego (np. pliku tekstowego)

Wprowadzanie danych

Tworzenie (wprowadzanie) macierzy:

Command Window

```
» A = [1 2 3; 4 5 6]
```

```
A =
```

```
1 2 3
```

```
4 5 6
```

Działanie operatora ':' jak np. w Fortranie:

Command Window

```
» x = [1 : 10]
```

```
x =
```

```
1 2 3 4 5 6 7 8 9 10
```

```
» x(5) = 0.1
```

```
x =
```

```
1.0000 2.0000 3.0000 4.0000 0.1000 6.0000 7.0000
```

```
8.0000 9.0000 10.0000
```

- Operator zakresu, elementy macierzy, wycinki
- Komendy **who**, **whos**
- Usuwanie zmiennych - **clear**, **clear all**
- Okno Workspace, Array editor
- Formaty danych - komenda **format xxx**
- Typy zmiennych

=	przypisanie
[]	tworzenie macierzy
()	element macierzy, argument funkcji
,	separator elementów wiersza, separator indeksów
.	separator dziesiętny, odwołanie do pola struktury
'	ogranicznik tekstu
;	separator kolumn, koniec linii
:	operator zakresu

Działania arytmetyczne

+	dodawanie
-	odejmowanie
*	mnożenie macierzy
.*	mnożenie macierzy wyraz po wyrazie
/	dzielenie prawostronne
./	dzielenie prawostronne wyraz po wyrazie
\	dzielenie lewostronne
.\	lewostronne wyraz po wyrazie
'	transpozycja sprzężona (sprzężenie hermitowskie) macierzy
.'	transpozycja macierzy

Uwaga

Działania w MATLAB można (i trzeba) stosować do całych macierzy.

Działania arytmetyczne

+	dodawanie
-	odejmowanie
*	mnożenie macierzy
.*	mnożenie macierzy wyraz po wyrazie
/	dzielenie prawostronne
./	dzielenie prawostronne wyraz po wyrazie
\	dzielenie lewostronne
.\	lewostronne wyraz po wyrazie
'	transpozycja sprzężona (sprzężenie hermitowskie) macierzy
.'	transpozycja macierzy

Uwaga

Działania w MATLAB można (i trzeba) stosować do całych macierzy.

- $\wedge$ | potęgowanie macierzy
- $\cdot \wedge$ | potęgowanie macierzy wyraz po wyrazie

- Operatory relacji:

`==, ~=, >, <, >=, <=`

- Operatory logiczne:

`&, |, ~, Xor, any, all, &&, ||`

Stałe i zmienne specjalne

ans	zmienna robocza przechowująca wynik ostatniego polecenia
eps	2^{-52}
inf	nieskończoność
NaN	Not-a-Number wartość nieokreślona
i, j	jednostka urojona
pi	liczba π
realmax	największa dostępna liczba rzeczywista
realmin	najmniejsza dostępna dodatnia liczba rzeczywista

Funkcje

MATLAB posiada ogromną liczbę wbudowanych funkcji, matematycznych i nie tylko. Większość funkcji potrafi przetwarzać macierze.

Funkcje działają na macierzach

```
» X = [0:pi/4:pi]
X =
0 0.7854 1.5708 2.3562 3.1416
» Y = sin(X)
Y =
0 0.7071 1.0000 0.7071 0.0000
```

Uwaga

Działania i funkcje MATLAB'a są zoptymalizowane pod kątem danych macierzowych zatem warto, a nawet należy używać macierzy zamiast pętli.

Funkcje

MATLAB posiada ogromną liczbę wbudowanych funkcji, matematycznych i nie tylko. Większość funkcji potrafi przetwarzać macierze.

Funkcje działają na macierzach

```
» X = [0:pi/4:pi]
X =
0 0.7854 1.5708 2.3562 3.1416
» Y = sin(X)
Y =
0 0.7071 1.0000 0.7071 0.0000
```

Uwaga

Działania i funkcje MATLAB'a są zoptymalizowane pod kątem danych macierzowych zatem warto, a nawet należy używać macierzy zamiast pętli.

Tematyczne zestawienie funkcji MATLAB'a:

- **help elfun** - podstawowe funkcje matematyczne
- **help elmat** - podstawowe funkcje macierzowe
- **help matfun** - funkcje macierzowe, algebra liniowa
- **help specfun** - specjalistyczne funkcje matematyczne
- **help polyfun** - funkcje interpolacyjne i wielomianowe
- **help datafun** - funkcje analizy danych i analizy fourierowskiej

Pomoc dla funkcji

help nazwa funkcji

MATLAB pozwala tworzyć własne pliki zawierające polecenia tzw. **M-pliki** (M-files). Są to pliki tekstowe .m zawierające:

- skrypty: zestawy wykonywanych po kolei komend
- funkcje: zdefiniowane przez użytkownika funkcje

Skrypty (m-pliki)

Przykładowy skrypt skrypt.m

```
%początek skryptu  
clear all  
clc  
disp('Początek działania skryptu.')  
x = [0:pi/10:2*pi];  
y = cos(x);  
plot(x,y) %wyświetla wykres z punktami x,y  
%koniec skryptu
```

Użycie

```
>> skrypt
```



Skrypty (m-pliki)

Przykładowy skrypt skrypt.m

```
%początek skryptu  
clear all  
clc  
disp('Początek działania skryptu.')  
x = [0:pi/10:2*pi];  
y = cos(x);  
plot(x,y) %wyświetla wykres z punktami x,y  
%koniec skryptu
```

Użycie

```
>> skrypt
```



Funkcje (M-pliki)

Przykładowa funkcja pierwiastki.m

```
function [x1, x2] = pierwiastki(a,b,c)
%%[x1,x2] = pierwiastki(a,b,c)
%%Oblicza pierwiastki równania  $a*x^2 + b*x + c = 0$ .
if( a == 0 )
    x1 = -c/b;
    x2 = x1;
else
    delta = b^2 - 4*a*c;
    x1 = (-b+sqrt(delta)) / (2*a);
    x2 = (-b-sqrt(delta)) / (2*a);
end
```

Użycie

```
>> [p1,p2] = pierwiastki(1,0,-1);
>> help pierwiastki;
```

Funkcje (M-pliki)

Przykładowa funkcja pierwiastki.m

```
function [x1, x2] = pierwiastki(a,b,c)
%%[x1,x2] = pierwiastki(a,b,c)
%%Oblicza pierwiastki równania  $a*x^2 + b*x + c = 0$ .
if( a == 0 )
    x1 = -c/b;
    x2 = x1;
else
    delta = b^2 - 4*a*c;
    x1 = (-b+sqrt(delta)) / (2*a);
    x2 = (-b-sqrt(delta)) / (2*a);
end
```

Użycie

```
>> [p1,p2] = pierwiastki(1,0,-1);
>> help pierwiastki;
```

Operacje na plikach

Podstawowe funkcje pozwalające:

- zapisywać dane do pliku: **save**
- odczytywać dane z pliku: **load**

Dane mogą być przechowywane w plikach **tekstowych** lub **binarnych** (MAT-plikach).

Zapis danych do pliku

```
>> save nazwapliku; % zapis wszystkich
% danych z pamięci do pliku binarnego
% nazwapliku.mat
>> save nazwapliku.dane x y -ascii;
% zapis tylko zmiennych
% x i y do pliku tekstowego nazwapliku.dane
```


Odczyt danych z pliku

```
>> load nazwapliku.mat;  
>> load nazwapliku.mat x y;  
% wczytanie tylko zmiennych x i y  
% z mat-pliku binarnego  
>> load -ascii nazwapliku.dane;  
% wczytanie zmiennych z pliku tekstowego
```

Zapis, wczytywanie i importowanie danych jest również możliwe z okienka przestrzeni roboczej (Workspace).

Język programowania środowiska MATLAB posiada podstawowe funkcje sterujące:

- instrukcje warunkowe **if**, **elseif**, **else**
- instrukcje **switch**, **case**, **otherwise**
- instrukcje pętli **for**, **while**, **continue**, **break**

Funkcje do pomiaru czasu obliczeń:

- pomiar czasu procesora **cputime**
- stoper **tic ... toc**

- Format dla danych numerycznych - polecenie **format**:
 - **short**: 5 cyfr, reprezentacja stałoprzecinkowa
 - **long**: 15 cyfr, reprezentacja stałoprzecinkowa
 - **short e**: 5 cyfr, reprezentacja zmiennoprzecinkowa
 - **long e**: 15 cyfr, reprezentacja zmiennoprzecinkowa
 - **short g**: maksymalnie 5 cyfr znaczących
 - **long g**: maksymalnie 15 cyfr znaczących
 - **hex**: liczba w układzie szesnastkowym
 - **bank**: 2 cyfry dziesiętne
 - **rat**: postać ułamka zwykłego (przybliżona)
- Komunikaty tekstowe - polecenie **disp**
- Funkcja języka C - **fprintf**
- Konwersje **num2str**, **str2num**

Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu**
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM



Optymalizowanie programów w MATLAB:

- **wektoryzacja** kodu
- **alokowanie** zmiennych
- kodowanie **pętli do plików MEX**
- zachowywanie typów istniejących zmiennych
- korzystanie z odpowiednich **predefiniowanych funkcji**
- użycie odpowiednich **operatorów logicznych**
- **wyższość funkcji** nad skryptami

Warto wektoryzować!

MATLAB jest środowiskiem zoptymalizowanym do operacji na macierzach i wektorach.

Najprostszym przykładem wektoryzacji kodu może być tworzenie wektora:

Tworzenie wektora w pętli

```
for i=1:1000,  
v(i) = i;  
end
```

Wersja wektorowa

```
v = [1:1000];
```

Warto wektoryzować!

MATLAB jest środowiskiem zoptymalizowanym do operacji na macierzach i wektorach.

Najprostszym przykładem wektoryzacji kodu może być tworzenie wektora:

Tworzenie wektora w pętli

```
for i=1:1000,  
v(i) = i;  
end
```

Wersja wektorowa

```
v = [1:1000];
```


Zalety wektoryzacji

- **Wydajniejszy kod** programu
- .. a zatem **krótszy czas** obliczeń
- Zwięzły i czytelny kod

Przykład: mnożenie macierzy 128x128

Metoda	Czas
$C(i,j) = C(i,j) + A(i,k)*B(k,j)$	0.182105 seconds
$C(i,j) = C(i,j) + A(i,:)*B(:,j)$	0.285265 seconds
$C = C + A*B$	0.008435 seconds

Zalety wektoryzacji

- **Wydajniejszy kod** programu
- .. a zatem **krótszy czas** obliczeń
- Zwięzły i czytelny kod

Przykład: mnożenie macierzy 128x128

Metoda	Czas
$C(i,j) = C(i,j) + A(i,k)*B(k,j)$	0.182105 seconds
$C(i,j) = C(i,j) + A(i,:)*B(:,j)$	0.285265 seconds
$C = C + A*B$	0.008435 seconds

- **Alokacja zmiennych**

zmienianie rozmiarów zmiennych jest dużym obciążeniem

- **Kodowanie pętli do plików MEX**

fragmenty kodu, które nie dają się zwektoryzować należy przenieść do tzw. pliku MEX, zewnętrznego prekompilowanego programu (np. w C lub Fortranie)

- **Zachowanie typów istniejących zmiennych**

Zmiana typu danych istniejącej już wcześniej zmiennej nie jest wydajne

- **Korzystanie z odpowiednich predefiniowanych funkcji**
Odpowiednie wykorzystanie funkcji w zależności od typu danych jest istotne dla wydajności
- **Użycie odpowiednich operatorów logicznych**
Należy stosować odpowiednie operatory ze względu na ich wydajność
- **Wyższość funkcji nad skryptami**
Kod zapisany w funkcji wykonuje się przeważnie znacznie szybciej niż kod zapisany w skrypcie

Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja**
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM

MATLAB posiada bardzo duże możliwości wizualizacji danych

- wykresy 2D
- wykresy 3D
- rysowanie pól wektorowych
- wizualizacja macierzy
- manipulacja i wizualizowanie obrazów
- wyresy zmienne w czasie i animacje vibes

Podstawowym elementem wizualizacyjnym w MATLAB'ie jest okno graficzne (Figure):

- nowe okno tworzy polecenie **figure**(numer).
- MATLAB automatycznie tworzy nowe okno lub korzysta z istniejącego aktywnego okna
- zamykanie istniejących okien - polecenie **close**
- funkcje **cla** i **clf** - czyszczenie bieżącego wykresu oraz do czyszczenie aktywnego okna

Do tworzenia dwuwymiarowych wykresów służą przede wszystkim funkcje **plot** i **fplot**.

- `plot(X,Y,LineSpec)` Przykład hndlgraf
- `fplot(funkcja, zakres, LineSpec)`

Parametry wykresu

Parametr **LineSpec** pozwala określić własności wykresu takie jak kolor, grubość, typ linii, znaczniki i ich typ.

Funkcje plot są bardzo elastyczne, potrafią narysować różne struktury.

demo matlab graphics

Wykresy można dostosowywać do własnych potrzeb, przy pomocy:

- **hold on, hold off** - blokowanie czyszczenia okna przy nowym wykresie Przykład
- **title** - dodaje tytuł do wykresu w aktywnym oknie Przykład
- **xlabel** - dodaje opis do osi x
- **ylabel** - dodaje opis do osi y
- **text** - dodaje tekst do wykresu w danym punkcie Przykład
- inne - **annotation, colorbar, legend**

MATLAB pozwala opisywać wykresy formułami matematycznymi w **LaTeX-u**. Można to zrobić np. przy pomocy komendy **text**:

Formuła LaTeX-a na wykresie MATLAB'a

title (

```
'A \cdot e^{\{-\alpha t\}} \sin \beta t'
```

)

pozwoili dodać napis $A \cdot e^{-\alpha t} \sin \beta t$ na wykresie

MATLAB posiada także gotowe funkcje do rysowania szczególnych wykresów, przykładowo:

- **bar**, **bar3** - wykresy słupkowe 2D i 3D
- **hist** - histogramy
- **stem**, **stairs** - wykresy schodkowe

Więcej informacji na temat specjalnych wykresów można znaleźć w systemie pomocy.

MATLAB, a w szczególności dodatek **Image Processing Toolbox**, umie przetwarzać obrazy:

- funkcja **imread**(plik) - pozwala wczytać obraz z pliku jako macierz i wyświetlić
 - funkcja **image** Przykład
 - funkcja **imagesc**
 - funkcja **imshow** (Image Processing Toolbox)

Wizualizacja macierzy

W szczególności można wizualizować dowolną macierz jako płaski obraz poleceniem **imagesc**. Przykład

MATLAB, a w szczególności dodatek **Image Processing Toolbox**, umie przetwarzać obrazy:

- funkcja **imread**(plik) - pozwala wczytać obraz z pliku jako macierz i wyświetlić
 - funkcja **image** Przykład
 - funkcja **imagesc**
 - funkcja **imshow** (Image Processing Toolbox)

Wizualizacja macierzy

W szczególności można wizualizować dowolną macierz jako płaski obraz poleceniem **imagesc**. Przykład

MATLAB posiada również kilka przydatnych funkcji tworzących wykresy trójwymiarowe:

- **plot3(X,Y,Z,LineSpec)** - rysuje krzywe w 3D, podobnie do plot
Przykład
- **mesh, surf, waterfall, meshc** - pozwalają rysować powierzchnie postaci $z = f(x,y)$ Przykład
- **contour** - rysuje wykres poziomic dla powierzchni graf3d

Wiele wykresów na jednym rysunku

W MATLAB można tworzyć wiele wykresów na jednym rysunku. Służy do tego polecenie **subplot**.

Wiele wykresów

subplot(m, n, p) wstawia wykres do rysunku-tablicy $m \times n$ w pozycji p .

Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne**
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 MATLAB w ICM

- Operatory arytmetyczne dla macierzy

+	suma
—	różnica
*	iloczyn macierzy
.*	mnożenie wyraz po wyrazie
^	potęgowanie macierzy
.^	potęgowanie wyraz po wyrazie
'	transpozycja sprzężona (sprzężenie hermitowskie)
.'	transpozycja macierzy

- Odwracanie macierzy
 - **inv()** - używa LAPACKa

Uwaga

W praktyce lepiej unikać jawnego odwracania macierzy; lepiej rozwiązać układ r-ń w celu znalezienia odwrotności za pomocą operatora / lub \.

- Odwracanie macierzy
 - **inv()** - używa LAPACKa

Uwaga

W praktyce lepiej unikać jawnego odwracania macierzy; lepiej rozwiązać układ r -ń w celu znalezienia odwrotności za pomocą operatora `/` lub `\`.

- Operacje na diagonalach
 - **diag(x)** - tworzy macierz diagonalną o diagonalu x
 - **diag(x, k)** - tworzy macierz o k-tej diagonalu x
 - **diag(A)** - wyznacza diagonalę macierzy A
- Inne operatory dla macierzy
 - **det(A)** - wyznacznik
 - **rank(A)** - rząd
 - **trace(A) = sum(diag(A))** - ślad
- Normy macierzy **norm(A,p)**
 - **p = 1** - 1-norma $\max(\text{sum}(\text{abs}(A)))$
 - **p = 2 (domyślne)** - 2-norma $\max(\text{eig}(A' * A))^{1/2}$
 - **p = inf** - $\max(\text{sum}(\text{abs}(A')))$
 - **p = 'fro'** - norma Frobeniusa $\sqrt{\text{sum}(\text{diag}(A' * A))}$

- Operacje na diagonalach
 - **diag(x)** - tworzy macierz diagonalną o diagonalu x
 - **diag(x, k)** - tworzy macierz o k-tej diagonalu x
 - **diag(A)** - wyznacza diagonalę macierzy A
- Inne operatory dla macierzy
 - **det(A)** - wyznacznik
 - **rank(A)** - rząd
 - **trace(A) = sum(diag(A))** - ślad
- Normy macierzy **norm(A,p)**
 - **p = 1** - 1-norma $\max(\text{sum}(\text{abs}(A)))$
 - **p = 2 (domyślne)** - 2-norma $\max(\text{eig}(A' * A))^{1/2}$
 - **p = inf** - $\max(\text{sum}(\text{abs}(A')))$
 - **p = 'fro'** - norma Frobeniusa $\text{sqrt}(\text{sum}(\text{diag}(A' * A)))$

- Operacje na diagonalach
 - **diag(x)** - tworzy macierz diagonalną o diagonalu x
 - **diag(x, k)** - tworzy macierz o k-tej diagonalu x
 - **diag(A)** - wyznacza diagonalę macierzy A
- Inne operatory dla macierzy
 - **det(A)** - wyznacznik
 - **rank(A)** - rząd
 - **trace(A) = sum(diag(A))** - ślad
- Normy macierzy **norm(A,p)**
 - **p = 1** - 1-norma $\max(\text{sum}(\text{abs}(A)))$
 - **p = 2 (domyślne)** - 2-norma $\max(\text{eig}(A' * A))^{\frac{1}{2}}$
 - **p = inf** - $\max(\text{sum}(\text{abs}(A')))$
 - **p = 'fro'** - norma Frobeniusa $\sqrt{\text{sum}(\text{diag}(A'*A))}$

- Zagadnienie własne
 - **lam = eig(A)** - oblicza wektor wartości własnych macierzy A
 - **[V, D] = eig(A)** - oblicza macierz wektorów własnych V i macierz diagonalną D o wartościach własnych A na diagonalu
 - **jordan(A)** - postać Jordana (tylko *Symbolic Toolbox*)
- Uwarunkowanie macierzy
 - **cond(A,p) = norm(A,p) * norm(inv(A),p)**
 - **rcond(A) = 1 \ norm(A,1)**
- EkspONENTA macierzy
 - **expm(A)** - oblicza e^A
 - **exp(A)** - oblicza expONENTę wyraz po wyrazie

- Zagadnienie własne
 - **lam = eig(A)** - oblicza wektor wartości własnych macierzy A
 - **[V, D] = eig(A)** - oblicza macierz wektorów własnych V i macierz diagonalną D o wartościach własnych A na diagonalu
 - **jordan(A)** - postać Jordana (tylko *Symbolic Toolbox*)
- Uwarunkowanie macierzy
 - **cond(A,p) = norm(A,p) * norm(inv(A),p)**
 - **rcond(A) = 1 \ norm(A,1)**
- EkspONENTA macierzy
 - **expm(A)** - oblicza e^A
 - **exp(A)** - oblicza expONENTę wyraz po wyrazie

- Zagadnienie własne
 - **lam = eig(A)** - oblicza wektor wartości własnych macierzy A
 - **[V, D] = eig(A)** - oblicza macierz wektorów własnych V i macierz diagonalną D o wartościach własnych A na diagonalu
 - **jordan(A)** - postać Jordana (tylko *Symbolic Toolbox*)
- Uwarunkowanie macierzy
 - **cond(A,p) = norm(A,p) * norm(inv(A),p)**
 - **rcond(A) = 1 \ norm(A,1)**
- EkspONENTA macierzy
 - **expm(A)** - oblicza e^A
 - **exp(A)** - oblicza expONENTę wyraz po wyrazie

- Rozkłady macierzy

- $[L,U] = \text{lu}(A)$ - rozkład trójkątno-trójkątny macierzy $A = LU$
- $[Q,R] = \text{qr}(A)$ - ortogonalno-górnotrójkątny macierzy $A = Q^*R$
- $[U,S,V] = \text{svd}(A)$ - wg. wartości szczególnych macierzy $A = U^*S^*V'$
- $R = \text{chol}(A)$ - rozkład Choleskiego macierzy $A = R'^*R$
- $[U,T] = \text{schur}(A)$ - rozkład Schura macierzy $A = U^*T^*U'$

- Macierze rozrzedzone i pasmowe

- $\text{nnz}(A)$ - liczba niezerowych wyrazów macierzy A
- $S = \text{sparse}(A)$ - tworzy macierz rozrzedzoną Przykład
- $A = \text{full}(S)$ - tworzy macierz pełną

- Obrazowanie macierzy

- $\text{spy}(A)$ - graficzny rozkład niezerowych elementów macierzy A
Przykład
- $\text{imagesc}(A)$ - wyświetla macierz A jako obraz

- Rozkłady macierzy

- $[L,U] = \text{lu}(A)$ - rozkład trójkątno-trójkątny macierzy $A = LU$
- $[Q,R] = \text{qr}(A)$ - ortogonalno-górnotrójkątny macierzy $A = Q^*R$
- $[U,S,V] = \text{svd}(A)$ - wg. wartości szczególnych macierzy $A = U^*S^*V'$
- $R = \text{chol}(A)$ - rozkład Choleskiego macierzy $A = R'^*R$
- $[U,T] = \text{schur}(A)$ - rozkład Schura macierzy $A = U^*T^*U'$

- Macierze rozrzedzone i pasmowe

- $\text{nnz}(A)$ - liczba niezerowych wyrazów macierzy A
- $S = \text{sparse}(A)$ - tworzy macierz rozrzedzoną Przykład
- $A = \text{full}(S)$ - tworzy macierz pełną

- Obrazowanie macierzy

- $\text{spy}(A)$ - graficzny rozkład niezerowych elementów macierzy A
Przykład
- $\text{imagesc}(A)$ - wyświetla macierz A jako obraz

- Rozkłady macierzy
 - $[L,U] = \text{lu}(A)$ - rozkład trójkątno-trójkątny macierzy $A = LU$
 - $[Q,R] = \text{qr}(A)$ - ortogonalno-górnotrójkątny macierzy $A = Q^*R$
 - $[U,S,V] = \text{svd}(A)$ - wg. wartości szczególnych macierzy $A = U^*S^*V'$
 - $R = \text{chol}(A)$ - rozkład Choleskiego macierzy $A = R'^*R$
 - $[U,T] = \text{schur}(A)$ - rozkład Schura macierzy $A = U^*T^*U'$
- Macierze rozrzedzone i pasmowe
 - $\text{nnz}(A)$ - liczba niezerowych wyrazów macierzy A
 - $S = \text{sparse}(A)$ - tworzy macierz rozrzedzoną Przykład
 - $A = \text{full}(S)$ - tworzy macierz pełną
- Obrazowanie macierzy
 - $\text{spy}(A)$ - graficzny rozkład niezerowych elementów macierzy A
Przykład
 - $\text{imagesc}(A)$ - wyświetla macierz A jako obraz

- Rozwiązanie układów równań postaci $Ax = b$
 - $x = A \setminus b$ - rozwiązuje układ $Ax = b$
 - $x = b / A$ - rozwiązuje układ $xA = b$
 - **null(A)** - rozwiązuje jednorodny układ równań z macierzą A
 - **linsolve(A, b)** - rozwiązuje układ za pomocą rozkładu LU dla macierzy A kwadratowej lub rozkładu QR w innych przypadkach
- Metody iteracyjne
 - $x = \text{pcg}(A,b)$ - metoda sprzężonych gradientów
 - $x = \text{gmres}(A,b)$ - metoda GMRes
 - Inne : **bicg**, **cgs**, **lsqr**, **qmr**

- Rozwiązanie układów równań postaci $Ax = b$
 - $x = A \setminus b$ - rozwiązuje układ $Ax = b$
 - $x = b / A$ - rozwiązuje układ $xA = b$
 - **null(A)** - rozwiązuje jednorodny układ równań z macierzą A
 - **linsolve(A, b)** - rozwiązuje układ za pomocą rozkładu LU dla macierzy A kwadratowej lub rozkładu QR w innych przypadkach
- Metody iteracyjne
 - $x = \text{pcg}(A,b)$ - metoda sprzężonych gradientów
 - $x = \text{gmres}(A,b)$ - metoda GMRes
 - Inne : **bicg**, **cgs**, **lsqr**, **qmr**

- Kwadratury

- **quad(f, a, b)** - wyznacza przybliżoną wartość $\int_a^b f(x)dx$ za pomocą kwadratury Simpsona
- **quadl(f, a, b)** - j.w. za pomocą adaptacyjnej kwadratury Lobatto
- **dblquad(f, xmin, xmax, ymin, ymax)** - przybliża $\iint f$
- **triplequad** - przybliża $\iiint f$

- Przybliżanie pochodnej funkcji
 - **diff(X)** - wyznacza różnice kolejnych elementów wektora X
 - **gradient(F)** - wyznacza przybliżony gradient dla dyskretnej funkcji 2 zmiennych F
 - **del2(U)** - wyznacza dyskretny laplasjan dla dyskretnej funkcji 2 zmiennych U
- Różniczkowanie symboliczne
 - **diff(f)** - symboliczna pochodna funkcji f (*Symbolic Toolbox*)

Równania różniczkowe

Metody dla równań **oden nn xx()**, gdzie **nn** - rząd metody; **xx** - opcjonalne własności [odeexamples](#) [odedemo](#)

- równania w postaci jawnej $x' = f(t, x)$
Składnia: **[T,X] = oden nn xx(f,tspan,x0)**
 - nie-sztywne
 - **ode45** - metoda Runge-Kutty(4,5) jednokrokowa
 - **ode23** - metoda Runge-Kutty(2,3) jednokrokowa
 - **ode113** - metoda wielokrokowa (Adams-Bashforth-Moulton)
 - sztywne
 - **ode15s** - metoda wielokrokowa (NDFs)
 - **ode23s** - metoda jednokroka, Rosenbrock formula of order 2
 - **ode23t**
 - **ode23tb**
- równania w postaci niejawnej $f(t, x, x') = 0$
[T,X] = ode15i(odefun,tspan,x0,xp0)

Równania różniczkowe

Metody dla równań **oden nn xx()**, gdzie **nn** - rząd metody; **xx** - opcjonalne własności [odeexamples](#) [odedemo](#)

- równania w postaci jawnej $x' = f(t, x)$
Składnia: **[T,X] = oden nn xx(f,tspan,x0)**
 - nie-sztywne
 - **ode45** - metoda Runge-Kutty(4,5) jednokrokowa
 - **ode23** - metoda Runge-Kutty(2,3) jednokrokowa
 - **ode113** - metoda wielokrokowa (Adams-Bashforth-Moulton)
 - sztywne
 - **ode15s** - metoda wielokrokowa (NDFs)
 - **ode23s** - metoda jednokroka, Rosenbrock formula of order 2
 - **ode23t**
 - **ode23tb**
- równania w postaci niejawnej $f(t, x, x') = 0$
[T,X] = ode15i(odefun,tspan,x0,xp0)

Równania różniczkowe

Metody dla równań **oden nn xx()**, gdzie **nn** - rząd metody; **xx** - opcjonalne własności `odeexamples` `odedemo`

- równania w postaci jawnej $x' = f(t, x)$
Składnia: **[T,X] = oden nn xx(f,tspan,x0)**
 - nie-sztywne
 - **ode45** - metoda Runge-Kutty(4,5) jednokrokowa
 - **ode23** - metoda Runge-Kutty(2,3) jednokrokowa
 - **ode113** - metoda wielokrokowa (Adams-Bashforth-Moulton)
 - sztywne
 - **ode15s** - metoda wielokrokowa (NDFs)
 - **ode23s** - metoda jednokroka, Rosenbrock formula of order 2
 - **ode23t**
 - **ode23tb**
- równania w postaci niejawnej $f(t, x, x') = 0$
[T,X] = ode15i(odefun,tspan,x0,xp0)

Równania różniczkowe

Metody dla równań **oden nn xx()**, gdzie **nn** - rząd metody; **xx** - opcjonalne własności [odeexamples](#) [odedemo](#)

- równania w postaci jawnej $x' = f(t, x)$
Składnia: **[T,X] = oden nn xx(f,tspan,x0)**
 - nie-sztywne
 - **ode45** - metoda Runge-Kutty(4,5) jednokrokowa
 - **ode23** - metoda Runge-Kutty(2,3) jednokrokowa
 - **ode113** - metoda wielokrokowa (Adams-Bashforth-Moulton)
 - sztywne
 - **ode15s** - metoda wielokrokowa (NDFs)
 - **ode23s** - metoda jednokroka, Rosenbrock formula of order 2
 - **ode23t**
 - **ode23tb**
- równania w postaci niejawnej $f(t, x, x') = 0$
[T,X] = ode15i(odefun,tspan,x0,xp0)

- Równania paraboliczne i eliptyczne w jednym wymiarze
pdepe(m, pdefun, icfun, bcfun, xmesh, tspan) - rozwiązuje równanie z warunkami początkowo-brzegowymi
- PDE (Partial Differential Equations) Toolbox
pdetool - graficzny interfejs Toolbox-a
- Przykłady: [pdex1](#) [pdex2](#) [pdex3](#) [pdex4](#) [pdex5](#)

- Przybliżone rozwiązywanie równań nieliniowych (*Optimization Toolbox*)
 - **$x = \text{fzero}(\text{fun}, x_0)$** - znajduje zera funkcji ciągłej z użyciem kombinacji metod bisekcji, secanta i int. kwadratowej
 - **$x = \text{fsolve}(\text{fun}, x_0)$** - rozwiązuje układ równań nieliniowych za pomocą alg. Gaussa-Newtona i innych

Interpolacja i wielomiany

- Dopasowywanie wielominanów
 - **basic fitting tool** - graficzne narzędzie dopasowywania wielomianów
 - **w = polyfit(x,y,n)** - wyznacza wsp. wielomianu w stopnia n: $w(x) = y$
 - **polytool** *Statistics Toolbox*
- Własności wielomianów
 - **w = poly(A)** - tworzy wektor wsp. wielomianu $w = \det(\lambda I - A)$
 - **w = poly(r)** - tworzy wektor wsp. wiel. w o pierw. w r
 - **r = roots(w)** - oblicza pierwiastki wielomianu o wsp. w
 - **polyder(w)** - oblicza pochodną wielomianu o wsp. w
 - **polyder(u,w)** - oblicza pochodną iloczynu wielomianów o wsp. u i w
 - **polyval(w,X)** - oblicza wartość wielomianu o wsp. w w punktach X
- Interpolacja
 - **interp1(x,Y,xi,method)** interpoluje wartości funkcji
 - **spline(x,Y)** - interpolacja splinem kubicznym
 - **pchip(x,y)** - interpolacja wielomianowa Hermite'a

Interpolacja i wielomiany

- Dopasowywanie wielominanów
 - **basic fitting tool** - graficzne narzędzie dopasowywania wielomianów
 - **w = polyfit(x,y,n)** - wyznacza wsp. wielomianu w stopnia n: $w(x) = y$
 - **polytool** *Statistics Toolbox*
- Własności wielomianów
 - **w = poly(A)** - tworzy wektor wsp. wielomianu $w = \det(\lambda I - A)$
 - **w = poly(r)** - tworzy wektor wsp. wiel. w o pierw. w r
 - **r = roots(w)** - oblicza pierwiastki wielomianu o wsp. w
 - **polyder(w)** - oblicza pochodną wielomianu o wsp. w
 - **polyder(u,w)** - oblicza pochodną iloczynu wielomianów o wsp. u i w
 - **polyval(w,X)** - oblicza wartosc wielomianu o wsp. w w punktach X
- Interpolacja
 - **interp1(x,Y,xi,method)** interpoluje wartości funkcji
 - **spline(x,Y)** - interpolacja splinem kubicznym
 - **pchip(x,y)** - interpolacja wielomianowa Hermite'a

Interpolacja i wielomiany

- Dopasowywanie wielominanów
 - **basic fitting tool** - graficzne narzędzie dopasowywania wielomianów
 - **w = polyfit(x,y,n)** - wyznacza wsp. wielomianu w stopnia n: $w(x) = y$
 - **polytool** *Statistics Toolbox*
- Własności wielomianów
 - **w = poly(A)** - tworzy wektor wsp. wielomianu $w = \det(\lambda I - A)$
 - **w = poly(r)** - tworzy wektor wsp. wiel. w o pierw. w r
 - **r = roots(w)** - oblicza pierwiastki wielomianu o wsp. w
 - **polyder(w)** - oblicza pochodną wielomianu o wsp. w
 - **polyder(u,w)** - oblicza pochodną iloczynu wielomianów o wsp. u i w
 - **polyval(w,X)** - oblicza wartość wielomianu o wsp. w w punktach X
- Interpolacja
 - **interp1(x,Y,xi,method)** interpoluje wartości funkcji
 - **spline(x,Y)** - interpolacja splinem kubicznym
 - **pchip(x,y)** - interpolacja wielomianowa Hermite'a

Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 **Toolbox-y**
- 8 **MATLAB w ICM**



Toolbox-y (czyli pakiety narzędzi) są modułami rozszerzającymi możliwości MATLAB'a o dodatkowe narzędzia. Jest ich około 50, między innymi:

- MATLAB Distributed Computing Toolbox
- MATLAB Distributed Computing Engine
- Optimization Toolbox
- **Symbolic Math Toolbox**
- **Extended Symbolic Math Toolbox**
- **Partial Differential Equation Toolbox**
- Genetic Algorithm and Direct Search Toolbox

- **Statistics Toolbox**
- Neural Network Toolbox
- Curve Fitting Toolbox
- Spline Toolbox

- Control System Toolbox
- Fuzzy Logic Toolbox
- Model Predictive Control Toolbox

- **Signal Processing Toolbox**

- Communications Toolbox
- Filter Design Toolbox
- Wavelet Toolbox
- Fixed-Point Toolbox

- **Image Processing Toolbox**

- Image Acquisition Toolbox
- Bioinformatics Toolbox
- Financial Toolbox
- Financial Derivatives Toolbox

- **MATLAB Compiler**

- Excel Link



Plan prezentacji

- 1 Wstęp do MATLAB'a
- 2 Interfejs graficzny MATLAB'a i Pomoc
- 3 Podstawy pracy z MATLAB'em
- 4 Optymalizacja kodu
- 5 Wizualizacja
- 6 Podstawowe problemy matematyczne
 - Algebra macierzy
 - Rozwiązywanie układów równań liniowych
 - Całkowanie numeryczne
 - Różniczkowanie
 - Równania różniczkowe zwyczajne
 - Równania różniczkowe cząstkowe
 - Równania nieliniowe
 - Interpolacja i wielomiany
- 7 Toolbox-y
- 8 **MATLAB w ICM**



MATLAB jest dostępny w ICM na następujących komputerach:

- **halo** - klaster obliczeniowy
- **rekin**
- **burza**
- **lab** - pracownia edukacyjna

Uruchamianie MATLAB'a

Aby uruchomić MATLAB'a na komputerach ICM, należy wykonać

```
use_matlab  
matlab
```

MATLAB w trybie wsadowym

Uruchamianie MATLAB'a w trybie tekstowym

Aby uruchomić MATLAB'a na konsoli tekstowej np. halo:

matlab -nosplash -nodesktop -nojvm

MATLAB'a można uruchomić trybie wsadowym

Uruchamianie MATLAB'a w systemie kolejkowym

```
#!/bin/csh
#PBS -S /bin/csh
#PBS -N matlab_test
#PBS -l cput=1:00
#PBS -l mem=1mb
#PBS -l ncpus=1

use_matlab
matlab -nojvm -nodesktop -nosplash < test.m
```


MATLAB w trybie wsadowym

Uruchamianie MATLAB'a w trybie tekstowym

Aby uruchomić MATLAB'a na konsoli tekstowej np. halo:

matlab -nosplash -nodesktop -nojvm

MATLAB'a można uruchomić trybie wsadowym

Uruchamianie MATLAB'a w systemie kolejkowym

```
#!/bin/csh
#PBS -S /bin/csh
#PBS -N matlab_test
#PBS -l cput=1:00
#PBS -l mem=1mb
#PBS -l ncpus=1
```

```
use_matlab
```

```
matlab -nojvm -nodesktop -nosplash < test.m
```

- W ICM: www.icm.edu.pl/kdm/programy/matlab/
- Producent: www.mathworks.com
- MATLAB Central: www.mathworks.com/matlabcentral/
- Podręcznik C.Moler'a: www.mathworks.com/moler/

Dziękujemy za uwagę